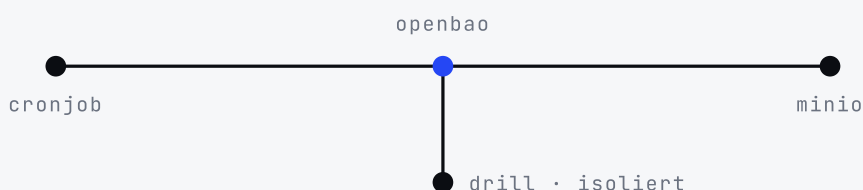


# RAFT- SNAPSHOTS OPENBAO

Ein CronJob mit eigener Identität, geprüfte Archive nach MinIO, und ein Restore, den man wirklich geprobt hat.



# Raft-Snapshots aus dem Cluster

---

Ein CronJob, der sich mit seinem ServiceAccount anmeldet, den Snapshot prüft und nach S3 schreibt – und eine Restore-Probe, die zeigt, warum dabei zwei Sätze Unseal-Keys gebraucht werden

Thomas Zachmann

17. September 2026

# Inhalt

## Worum es geht

- Warum von Hand, und warum ohne KI
- Lizenz und Haftung
- Ein Wort zu den Werten in diesem Leitfaden
- Für wen
- Die Bausteine
- Die Architektur in einem Bild

## Teil I – Das Ziel: MinIO

- Deployment
- Lifecycle statt Aufräum-Skript

## Teil II – Der Snapshot-Job

- Wer darf: Policy und Auth-Rolle
- Der Job
- Der erste Lauf – und der stille Fehler
- Der Staleness-Check

## Teil III – Die Restore-Probe

- Warum isoliert
- Die Instanz
- Das, was alle falsch machen
- Die Probe, selbst-enthalten
- Die Probe mit dem echten Snapshot
- Was die Probe nicht beweist

## Teil IV – Was schiefging, und warum

## Teil V – Betrieb

- Prüfen, dass es läuft
- Keys getrennt von Snapshots
- Was noch offen ist

## Anhang A – Alle Kommandos

## Anhang B – Glossar

# Worum es geht

---

N° 1 endete mit einem unbequemen Satz: Das PVC ist die einzige Kopie der Raft-Daten. Longhorn repliziert das Volume über Nodes – das schützt gegen eine tote Platte, nicht gegen ein `helm uninstall` mit gelöschtem PVC, ein kaputtes Upgrade oder einen Fehlgriff mit `bao delete`. Alles, was in diesem OpenBao liegt – KV-Secrets, die Datenbank-Rollen aus N° 2, die CA aus N° 5 – hängt an diesem einen Volume.

Dieser Leitfaden baut das Backup: ein CronJob, der sich bei OpenBao mit seinem ServiceAccount anmeldet, `bao operator raft snapshot save` ausführt, das Archiv prüft und nach MinIO hochlädt. Ein zweiter CronJob, der Alarm schlägt, wenn der jüngste Snapshot älter als 26 Stunden ist. Und – der wichtigste Teil – eine Restore-Probe auf eine isolierte zweite Instanz, die den Punkt sichtbar macht, den jeder beim ersten Mal falsch versteht: Ein Restore auf einen frischen Cluster braucht **zwei** Sätze Unseal-Keys, nacheinander.

Es ist der siebte Teil einer Reihe. N° 3 beschreibt dasselbe auf einer VM mit systemd-Timer und NFS; die Verifikation und der Restore-Ablauf sind von dort übernommen. Der Cluster-Weg ist an einer Stelle besser: Es gibt keinen periodischen Token in einer Datei – der Job meldet sich bei jedem Lauf neu an.

Alles wurde am Cluster durchgeführt. Vier Fehler sind drin, einer davon war ein stiller.

## Warum von Hand, und warum ohne KI

„Schreib mir einen CronJob, der OpenBao nach S3 sichert“ liefert in Sekunden ein Manifest, das läuft. Es liefert nicht: warum der Job Erfolg meldete, obwohl seine Retention nie funktioniert hat; warum das `mc`-Image kein `grep` hat und was das mit `set -e` macht; warum `kubectl cp` in diesem Container scheitert; und warum ein Restore ohne `-force` abgelehnt wird und mit `-force` die Instanz versiegelt zurücklässt.

Werkzeuge wie [nyrvex](#), das die Konfiguration von Secret Store und Identity Provider einer AI-Plattform generiert, nehmen einem diese Schritte später ab. Man sollte sie einmal selbst gegangen sein, um beurteilen zu können, was generiert wurde.

Der Maßstab: Wer diesen Leitfaden durchgearbeitet hat, kann auf einem leeren Blatt aufzeichnen, welche vier Schritte ein Restore auf einen neuen Cluster hat, welcher Key-Satz in welchem Schritt gilt, und warum die Probe-Instanz kein Netzwerk haben darf.

## Lizenz und Haftung

Dieser Leitfaden steht unter CC BY 4.0: Er darf kopiert, weitergegeben und bearbeitet werden, auch kommerziell, solange der Autor genannt wird. Er wird ohne Gewähr bereitgestellt. Alles darin wurde in einer Entwicklungsumgebung durchgeführt; wer es in einer anderen Umgebung nachvollzieht, tut das auf eigene Verantwortung.

# Ein Wort zu den Werten in diesem Leitfaden

MinIO-Zugangsdaten (minioadmin123, openbao-snapshot-123) sind Entwicklungswerte und bewusst abgedruckt. Unseal-Keys und Root-Tokens erscheinen nirgends – auch die der Probe-Instanz nicht, sie lebten nur in Shell-Variablen und sind mit der Shell verschwunden.

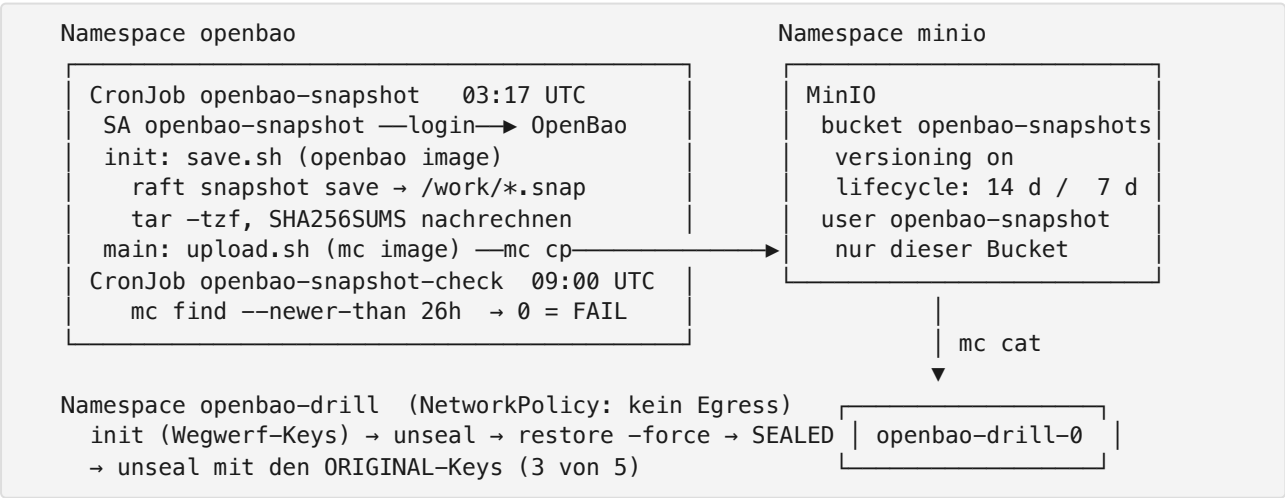
## Für wen

Für Leserinnen und Leser, die N° 1 kennen und wissen, was ein CronJob ist. N° 3 ist hilfreich, aber nicht Voraussetzung – die Restore-Logik wird hier noch einmal erklärt. OpenBao-Grundlagen liefert mein Buch **Vault in Practice** (Leanpub, [leanpub.com/vault-in-practice](https://leanpub.com/vault-in-practice)).

## Die Bausteine

| Baustein                  | Version                   | Rolle                                                  |
|---------------------------|---------------------------|--------------------------------------------------------|
| OpenBao im Cluster (N° 1) | 2.6.2                     | die Quelle; Kubernetes-Auth mit Rolle snapshot         |
| MinIO (Chart minio/minio) | 5.4.0, RELEASE.2024-12-18 | S3-Ziel, Bucket mit Versionierung und Lifecycle        |
| mc                        | RELEASE.2024-11-21        | Upload im Job, Lifecycle-Regel, Staleness-Check        |
| OpenBao-Image             | 2.6.2 (Alpine)            | Snapshot und Prüfung im Job – hat tar, sha256sum, wget |
| Zweite OpenBao-Instanz    | Chart 0.29.4              | Restore-Probe, Namespace openbao-drill, ohne Egress    |

## Die Architektur in einem Bild



Drei Dinge, die man aus dem Bild mitnehmen sollte:

1. **Zwei Container, eine Aufgabe.** Das OpenBao-Image kann bao und die Prüfung, das mc-Image kann S3. Ein Init-Container sichert, der Haupt-Container lädt hoch; scheitert der erste, läuft der zweite nie.
2. **Retention gehört dem Bucket, nicht dem Job.** Eine Lifecycle-Regel löscht nach 14 Tagen, alte Versionen nach 7. Der Job weiß davon nichts – und kann sie deshalb auch nicht kaputtmachen.
3. **Die Probe-Instanz ist ein Gefangener.** Ein wiederhergestellter OpenBao hält jede Lease des Originals. Erreicht er die Datenbank, kann er echte User dropen. Darum die NetworkPolicy, bevor der erste Snapshot hineinkommt.

# Teil I – Das Ziel: MinIO

---

## Deployment

Ein Standalone-MinIO mit einem PVC reicht als Ziel im selben Cluster – das ist keine Offsite-Kopie, aber es trennt die Snapshots vom PVC des OpenBao. Wer Longhorn-Backups nach extern hat, bekommt die zweite Ebene damit mit; ein externes S3 wäre ein Wechsel der URL im Secret.

```
mode: standalone
persistence:
  size: 20Gi
  storageClass: longhorn
rootUser: minioadmin
rootPassword: minioadmin123

buckets:
- name: openbao-snapshots
  versioning: true # ein überschriebener Snapshot bleibt wiederherstellbar
policies:
- name: openbao-snapshots-rw
  statements:
  - resources: ["arn:aws:s3:::openbao-snapshots"]
    actions: ["s3:ListBucket", "s3:GetBucketLocation"]
  - resources: ["arn:aws:s3:::openbao-snapshots/*"]
    actions: ["s3:PutObject", "s3:GetObject", "s3:DeleteObject"]
users:
- accessKey: openbao-snapshot
  secretKey: openbao-snapshot-123
  policy: openbao-snapshots-rw
```

```
helm repo add minio https://charts.min.io/
helm upgrade --install minio minio/minio --version 5.4.0 \
  -n minio --create-namespace -f k8s/minio-values.yaml
```

Das Chart legt Bucket, Policy und Benutzer über einen Post-Install-Job an. Prüfen, mit dem mc-Image des Charts selbst:

```
$ kubectl run -n minio --rm -i --restart=Never --image=quay.io/minio/mc:RELEASE.2024-11-21
T17-21-54Z mc --command -- sh -c '
  mc alias set local http://minio:9000 minioadmin minioadmin123
  mc version info local/openbao-snapshots
  mc admin user list local
  mc alias set snap http://minio:9000 openbao-snapshot openbao-snapshot-123
  echo test | mc pipe snap/openbao-snapshots/probe.txt && mc rm snap/openbao-snapshots/p
robe.txt
  mc mb snap/other'
local/openbao-snapshots versioning is enabled
enabled      openbao-snapshot      openbao-snapshots-rw
Created delete marker `snap/openbao-snapshots/probe.txt` ...
mc: <ERROR> Unable to make bucket `snap/other`. Access Denied.
```

Der Snapshot-Benutzer kann in seinen Bucket schreiben und sonst nichts. Die Access Denied ist der Test, der zählt.

Zwei Kleinigkeiten, die Zeit kosten: Das Image heißt `quay.io/minio/mc`, nicht `minio/mc` auf Docker Hub – der Tag dort existiert nicht mehr, und `kubectl run` hängt dann in `ImagePullBackOff`. Und `mc` warnt bei jedem Aufruf, dass Kommandos samt Credentials im Container-Log landen – für den Job unten sind die Credentials deshalb in der Umgebung (`MC_HOST_s3`), nicht in der Kommandozeile.

## Lifecycle statt Aufräum-Skript

```
$ mc ilm rule add --expire-days 14 --noncurrent-expire-days 7 local/openbao-snapshots
Lifecycle configuration rule added with ID `daltkkk79e2s76cvv84g`
```

Aktuelle Objekte verfallen nach 14 Tagen, ältere Versionen nach 7. Warum das serverseitig sein muss und nicht im Job, zeigt Teil II.

## Teil II – Der Snapshot-Job

---

### Wer darf: Policy und Auth-Rolle

```
/ $ bao policy write snapshot - <<'EOF'
path "sys/storage/raft/snapshot" { capabilities = ["read"] }
EOF
/ $ bao write auth/kubernetes/role/snapshot \
    bound_service_account_names=openbao-snapshot \
    bound_service_account_namespaces=openbao \
    token_policies=snapshot token_ttl=15m
```

Eine Zeile Policy, ein Pfad, read. Der Token lebt 15 Minuten – länger braucht kein Snapshot. Das ist der Unterschied zur VM (Nº 3): Dort liegt ein periodischer Token in `/etc/openbao/snapshot.token`, weil der systemd-Timer keine Identität hat. Ein Pod hat eine.

## Der Job

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: openbao-snapshot
  namespace: openbao
spec:
  schedule: "17 3 * * *"
  concurrencyPolicy: Forbid
  jobTemplate:
    spec:
      backoffLimit: 1
      template:
        spec:
          serviceAccountName: openbao-snapshot
          restartPolicy: Never
          volumes:
            - name: work
              emptyDir: {}
            - name: scripts
              configMap:
                name: openbao-snapshot-scripts
                defaultMode: 0755
          initContainers:
            - name: save
              image: quay.io/openbao/openbao:2.6.2
              command: ["/scripts/save.sh"]
              volumeMounts:
                - { name: work, mountPath: /work }
                - { name: scripts, mountPath: /scripts }
          containers:
            - name: upload
              image: quay.io/minio/mc:RELEASE.2024-11-21T17-21-54Z
              command: ["/scripts/upload.sh"]
              envFrom:
                - secretRef:
                    name: openbao-snapshot-s3
              volumeMounts:
                - { name: work, mountPath: /work }
                - { name: scripts, mountPath: /scripts }
```

save.sh – Login, Snapshot, Prüfung:

```
#!/bin/sh
set -eu
export BAO_ADDR=http://openbao.openbao.svc:8200
JWT=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)
BAO_TOKEN=$(bao write -field=token auth/kubernetes/login role=snapshot jwt="$JWT")
export BAO_TOKEN
TS=$(date -u +%Y%m%dT%H%M%SZ)
OUT=/work/openbao-$TS.snap
bao operator raft snapshot save "$OUT"
# Ein Raft-Snapshot ist ein gzipped tar mit meta.json, state.bin und
# SHA256SUMS. Nachrechnen - OpenBao hat kein 'snapshot inspect'.
tar -tzf "$OUT" >/dev/null
tar -xzf "$OUT" SHA256SUMS > /work/expected
for f in meta.json state.bin; do
    have=$(tar -xzf "$OUT" "$f" | sha256sum | cut -d' ' -f1)
    want=$(grep " $f\$" /work/expected | cut -d' ' -f1)
    [ "$have" = "$want" ] || { echo "integrity check failed for $f"; rm -f "$OUT"; exit 1; }
done
echo "snapshot ok: $OUT ($(wc -c < "$OUT") bytes)"
bao token revoke -self >/dev/null 2>&1 || true
```

upload.sh – nur noch hochladen:

```
#!/bin/sh
set -eu
f=$(ls /work/openbao-*.snap)
mc cp "$f" s3/openbao-snapshots/
echo "uploaded $(basename "$f"); bucket now:"
mc ls s3/openbao-snapshots/
```

Die MinIO-Zugangsdaten kommen aus einem Secret als MC\_HOST\_s3 – eine URL mit eingebetteten Credentials, die mc als Alias s3 versteht. Für die Demo ist das Secret ein Manifest; in einem echten Setup käme es per ExternalSecret aus dem KV-Engine (N° 4).

## Der erste Lauf – und der stille Fehler

```
$ kubectl create job -n openbao --from=cronjob/openbao-snapshot snapshot-manual-1
$ kubectl get job -n openbao snapshot-manual-1
NAME                STATUS      COMPLETIONS   DURATION
snapshot-manual-1   Complete    1/1           19s

$ kubectl logs -n openbao job/snapshot-manual-1 -c save
snapshot ok: /work/openbao-20260917T121637Z.snap (64540 bytes)

$ kubectl logs -n openbao job/snapshot-manual-1 -c upload
`/work/openbao-20260917T121637Z.snap` -> `s3/openbao-snapshots/openbao-20260917T121637Z.sn
ap`
/scripts/upload.sh: line 6: grep: command not found
uploaded openbao-20260917T121637Z.snap; bucket now:
[2026-09-17 12:16:39 UTC] 63KiB STANDARD openbao-20260917T121637Z.snap
```

Der Job ist Complete, der Snapshot liegt im Bucket – und in der Mitte steht grep: command not found. Die erste Fassung von upload.sh hatte eine Retention-Pipeline (mc ls --json | grep | cut | sort | head | while ... mc rm). Das mc-Image hat kein grep, kein sed, kein awk. Und set -e hat

nicht geholfen: In einer Pipeline zählt nur der Exit-Code des **letzten** Befehls, und head war zufrieden.

Ein Backup-Job, der Erfolg meldet, während ein Teil seiner Arbeit nie passiert, ist der gefährlichste Fehler in diesem Leitfaden. Er wäre erst aufgefallen, wenn der Bucket voll ist. Zwei Konsequenzen:

- Retention raus aus dem Job, rein in die Lifecycle-Regel (Teil I). Was der Job nicht tut, kann er nicht still falsch tun.
- Ein Job-Skript darf nur Befehle enthalten, die im Image existieren. Was drin ist, prüft man vorher:  
for t in grep sed awk cut sort head; do command -v \$t || echo "\$t MISSING"; done.

## Der Staleness-Check

Der Job selbst kann nicht melden, dass er nicht gelaufen ist. Ein zweiter CronJob prüft das **Ergebnis** – nicht den Timer:

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: openbao-snapshot-check
  namespace: openbao
spec:
  schedule: "0 9 * * *"
  jobTemplate:
    spec:
      backoffLimit: 0
      template:
        spec:
          restartPolicy: Never
          containers:
            - name: check
              image: quay.io/minio/mc:RELEASE.2024-11-21T17-21-54Z
              envFrom:
                - secretRef:
                    name: openbao-snapshot-s3
              command:
                - sh
                - -c
                - |
                  set -eu
                  n=$(mc find s3/openbao-snapshots --name "openbao-*.snap" --newer-than 26
h | wc -l)
                  echo "snapshots younger than 26h: $n"
                  [ "$n" -gt 0 ] || { echo "STALE: no recent snapshot"; exit 1; }
```

```
$ kubectl logs -n openbao job/check-manual-1
snapshots younger than 26h: 1
```

Und der Gegenteilstest mit `--newer-than 1s`: count=0, STALE, Exit 1. Ein fehlgeschlagener Job ist in `kubectl get jobs` sichtbar und – wenn es ein Monitoring gibt – als `kube_job_failed` in Prometheus. Hier gibt es keins; das ist die offene Stelle in Teil VII.

## Teil III – Die Restore-Probe

---

### Warum isoliert

Ein wiederhergestellter OpenBao ist nicht leer. Er hält jede Lease des Originals: die dynamischen Datenbank-User aus N° 2, die Tokens, die Zertifikate. Läuft er entsiegelt und erreicht die Datenbank, führt er beim Ablauf einer Lease DROP ROLE aus – gegen die **echte** Datenbank, für einen User, den das Original noch benutzt. Darum vor dem ersten Snapshot:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: drill-isolation
  namespace: openbao-drill
spec:
  podSelector: {}
  policyTypes: [Ingress, Egress]
  ingress:
    - from:
        - podSelector: {}
  egress:
    - to:
        - namespaceSelector:
            matchLabels:
                kubernetes.io/metadata.name: kube-system
  ports:
    - port: 53
      protocol: UDP
```

Kein Egress außer DNS. `kubectl exec` braucht keinen Netzwerkpfad in den Pod, das geht über den Kubelet. Der Beweis:

```
$ kubectl exec -n openbao-drill openbao-drill-0 -- wget -q0- -T 3 http://openbao.openbao.svc:8200/v1/sys/health
wget: download timed out
```

### Die Instanz

Ein zweites Helm-Release mit denselben Values wie N° 1, minus Ingress und UI, 2 Gi PVC:

```
helm upgrade --install openbao-drill openbao/openbao --version 0.29.4 \
  -n openbao-drill -f k8s/drill-values.yaml
```

### Das, was alle falsch machen

Ein Restore auf einen **neuen** Cluster braucht zwei Sätze Unseal-Keys:

1. Neue Instanz: init -> NEUE Keys, NEUER Root-Token
2. Unseal mit den NEUEN Keys -> läuft, aber leer
3. restore -force -> Keyring aus dem Snapshot ersetzt den neuen
4. Unseal mit den ORIGINAL-Keys -> die neuen sind wertlos

-force ist nötig, weil OpenBao prüft, ob der Snapshot zum eigenen Keyring passt. Tut er nicht – anderer Cluster, andere Keys – und ohne das Flag verweigert es genau deshalb. Die Keys aus Schritt 1 sind Wegwerf-Material für die Minuten zwischen Schritt 2 und 3.

## Die Probe, selbst-enthalten

Um den Ablauf zu beweisen, ohne die Produktions-Keys zu brauchen: Die Drill-Instanz spielt erst das Original (A), dann den neuen Cluster (B).

```
== A: init (1 Share), unseal, Marker schreiben, Snapshot
$ bao operator init -key-shares=1 -key-threshold=1
$ bao operator unseal <key A>
$ bao secrets enable -path=secret kv-v2
$ bao kv put secret/drill/marker value=written-before-snapshot
$ bao operator raft snapshot save /tmp/drillA.snap          # 19996 bytes

== Daten löschen, Pod neu -> nicht initialisiert
$ rm -rf /openbao/data/*; kubectl delete pod openbao-drill-0
Initialized    false

== B: init -> NEUE Keys
$ bao operator init -key-shares=1 -key-threshold=1
$ bao operator unseal <key B>
Sealed         false

== Restore OHNE -force:
$ bao operator raft snapshot restore /tmp/drillA.snap
* could not verify hash file, possibly the snapshot is using a different set
  of unseal keys; use the snapshot-force API to bypass this check

== Restore MIT -force:
$ bao operator raft snapshot restore -force /tmp/drillA.snap
$ bao status | grep Sealed
Sealed         true

== Unseal mit Key B:
Code: 400      <- abgelehnt

== Unseal mit Key A:
Sealed         false

== Marker da? (mit Root-Token A)
$ bao kv get -field=value secret/drill/marker
written-before-snapshot

== Root-Token B?
* permission denied
```

Jede Zeile davon ist ein Beleg: Ohne `-force` Ablehnung mit klarer Meldung. Mit `-force` versiegelt. Der neue Key wird abgelehnt, der alte entsiegelt. Die Daten sind die des Snapshots, inklusive Root-Token. Der Root-Token der Zwischeninstanz ist tot.

Ein Detail am Rand: `bao operator raft snapshot restore` gibt am Ende `Error properly closing policy file: ... file already closed` aus. Das ist ein kosmetischer Fehler der CLI, kein Problem des Restores – `bao status` danach ist die Wahrheit.

## Die Probe mit dem echten Snapshot

Dann derselbe Ablauf mit dem Produktions-Snapshot aus MinIO:

```
$ kubectl run -n minio --restart=Never --image=quay.io/minio/mc:... mcbox --command -- sleep 600
$ kubectl exec -n minio mcbox -- mc cat s3/openbao-snapshots/openbao-20260917T121637Z.snap > prod.snap
$ tar -tzf prod.snap
meta.json state.bin SHA256SUMS SHA256SUMS.sealed
$ tar -xzOf prod.snap state.bin | sha256sum          # stimmt mit SHA256SUMS überein

$ kubectl cp prod.snap openbao-drill/openbao-drill-0:/tmp/prod.snap
(drill: rm -rf data, init, unseal mit eigenem Key)
$ bao operator raft snapshot restore -force /tmp/prod.snap
$ bao status | grep -E 'Sealed|Total|Threshold'
Sealed          true
Total Shares    5
Threshold       3
```

Total Shares 5, Threshold 3 – das ist der Keyring des Produktions-OpenBao, nicht der 1/1 der Drill-Instanz. Ab hier braucht es drei der fünf echten Unseal-Keys, und die gehören in ein Terminal, nicht in einen Leitfaden. Die Instanz bleibt versiegelt stehen; entsiegelt sieht man darin alle Secrets aus N° 2, 4 und 5 – und kann sie gefahrlos anschauen, weil die NetworkPolicy hält.

Zwei Werkzeug-Fallen auf dem Weg, beide echt:

- `kubectl run --rm ... > datei` mischt die Meldung `pod "..." deleted` in die Datei. Für Binärdaten: Pod ohne `--rm` starten, `kubectl exec ... mc cat` umleiten, Pod danach löschen.
- `kubectl cp` braucht `tar` **im Container**. Das `mc`-Image hat keins; `kubectl cp` scheitert dann still oder mit `tar: not found`. Der Weg über `exec` und `stdout` ist robuster.

## Was die Probe nicht beweist

Dass die drei Keys funktionieren. Das kann nur, wer sie hat – und sollte es einmal tun, solange die Probe-Instanz steht. Danach:

```
kubectl delete namespace openbao-drill
```

## Teil IV – Was schiefging, und warum

---

| Fehler                 | Symptom                                                           | Ursache                                                                   | Lösung                                                                      |
|------------------------|-------------------------------------------------------------------|---------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| Stille Retention       | Job Complete, im Log<br>grep: command not<br>found                | mc-Image ohne<br>grep/sed/awk; Pipeline-<br>Exit-Code ist der von<br>head | Retention als Lifecycle-<br>Regel; Skript nur mit<br>Befehlen aus dem Image |
| Image nicht gefunden   | kubectl run hängt                                                 | minio/mc auf Docker Hub<br>hat den Tag nicht mehr                         | quay.io/minio/mc, Tag<br>aus dem Chart                                      |
| Restore verweigert     | could not verify hash<br>file ... different set<br>of unseal keys | Snapshot stammt aus<br>einem anderen Keyring                              | -force – und wissen,<br>dass danach die Original-<br>Keys gelten            |
| Kaputte Snapshot-Datei | Unrecognized archive<br>format                                    | kubectl run --rm hat<br>„pod deleted“ angehängt                           | kubectl exec ... mc cat<br>statt run --rm                                   |
| kubectl cp scheitert   | leere Datei                                                       | kein tar im mc-Container                                                  | Daten über stdout holen                                                     |

Und einer, der mir passiert ist und in keinem Manifest steht: Die Keys der Probe-Instanz lebten in Shell-Variablen, die zwischen zwei Aufrufen verloren gingen – der Restore war fertig, aber der Key zum Entsiegeln weg. Für eine Wegwerf-Instanz egal; für die echte wäre es das Ende. Unseal-Keys gehören in den Passwort-Manager, **bevor** man sie benutzt.

# Teil V – Betrieb

---

## Prüfen, dass es läuft

```
kubectl get cronjob -n openbao           # LAST SCHEDULE
kubectl get jobs -n openbao              # Complete / Failed
kubectl logs -n openbao job/<name> -c save
kubectl logs -n openbao job/<name> -c upload
```

Der Check-Job ist der, den man beobachten muss: Ein Failed dort heißt, seit 26 Stunden kein Snapshot.

## Keys getrennt von Snapshots

Die Snapshots sind mit dem Master-Key verschlüsselt, den die Unseal-Keys rekonstruieren. Liegen die Keys im selben MinIO, im selben Cluster, auf demselben Host wie die Snapshots, nimmt ein Verlust beides mit. Die Keys gehören in den Passwort-Manager; MinIO gehört, wenn es ernst wird, auf ein anderes Gerät oder in ein externes S3.

## Was noch offen ist

- **Kein Alarm.** Ein fehlgeschlagener Check-Job steht in `kubectl get jobs`, mehr nicht. Mit Prometheus/kube-state-metrics: `kube_job_failed` auf `openbao-snapshot-check`. Ohne: ein `mc`-Aufruf im Check, der eine Nachricht schickt.
- **MinIO im selben Cluster.** Ein Cluster-Verlust nimmt die Snapshots mit. Der nächste Schritt ist Replikation des Buckets nach extern (`mc replicate`) oder ein externes S3 als Ziel.
- **Die echte Probe.** Bis jemand die drei Keys in die Drill-Instanz eingegeben hat, ist der Produktions-Restore eine Annahme.
- **Kein Audit-Device** (N° 1). Wer wann einen Snapshot gezogen hat, steht nur in den Job-Logs.

# Anhang A – Alle Kommandos

---

```
# — MinIO —
helm upgrade --install minio minio/minio --version 5.4.0 -n minio --create-namespace -f k8s/minio-values.yaml
mc ilm rule add --expire-days 14 --noncurrent-expire-days 7 local/openbao-snapshots

# — OpenBao —
bao policy write snapshot - <<'EOF'
path "sys/storage/raft/snapshot" { capabilities = ["read"] }
EOF
bao write auth/kubernetes/role/snapshot bound_service_account_names=openbao-snapshot \
bound_service_account_namespaces=openbao token_policies=snapshot token_ttl=15m

# — Jobs —
kubectl apply -f k8s/snapshot-cronjob.yaml -f k8s/snapshot-check-cronjob.yaml
kubectl create job -n openbao --from=cronjob/openbao-snapshot snapshot-manual-1
kubectl logs -n openbao job/snapshot-manual-1 -c save
kubectl logs -n openbao job/snapshot-manual-1 -c upload
kubectl create job -n openbao --from=cronjob/openbao-snapshot-check check-manual-1

# — Restore-Probe —
kubectl create ns openbao-drill && kubectl apply -f k8s/drill-networkpolicy.yaml
helm upgrade --install openbao-drill openbao/openbao --version 0.29.4 -n openbao-drill -f k8s/drill-values.yaml
kubectl exec -n minio mcbox -- mc cat s3/openbao-snapshots/<datei> > prod.snap
kubectl cp prod.snap openbao-drill/openbao-drill-0:/tmp/prod.snap

# im Drill-Pod:
bao operator init -key-shares=1 -key-threshold=1          # Wegwerf-Keys
bao operator unseal <wegwerf-key>
bao operator raft snapshot restore -force /tmp/prod.snap
bao operator unseal                                       # 3x ORIGINAL-Keys
kubectl delete namespace openbao-drill                  # danach
```

# Anhang B – Glossar

---

**Raft-Snapshot** – Konsistenter Abzug des Raft-Storage: gzipped tar mit `meta.json`, `state.bin`, `SHA256SUMS` (und `SHA256SUMS.sealed`). Verschlüsselt mit dem Master-Key des Clusters, aus dem er stammt.

**Master-Key / Unseal-Keys** – Der Master-Key verschlüsselt den Storage; Shamir teilt ihn in Unseal-Keys. Ein Snapshot bringt seinen Keyring mit.

**-force** – Umgeht die Prüfung, ob der Snapshot zum aktuellen Keyring passt. Nötig bei Restore auf einen anderen Cluster; danach gelten die Keys des Snapshots.

**Init-Container** – Läuft vor den Haupt-Containern; scheitert er, startet der Pod nicht weiter. Hier: Snapshot und Prüfung.

**Lifecycle-Regel** – Serverseitige Retention in S3/MinIO: Objekte und alte Versionen verfallen nach Tagen.

**MC\_HOST\_<alias>** – Umgebungsvariable, mit der mc einen Alias samt Credentials kennt, ohne sie auf der Kommandozeile zu haben.

**Staleness-Check** – Prüfung des Ergebnisses (jüngstes Backup) statt des Auslösers (Timer lief).

**NetworkPolicy** – Kubernetes-Objekt, das Ingress/Egress eines Pods einschränkt. Hier: kein Egress außer DNS für die Probe-Instanz.

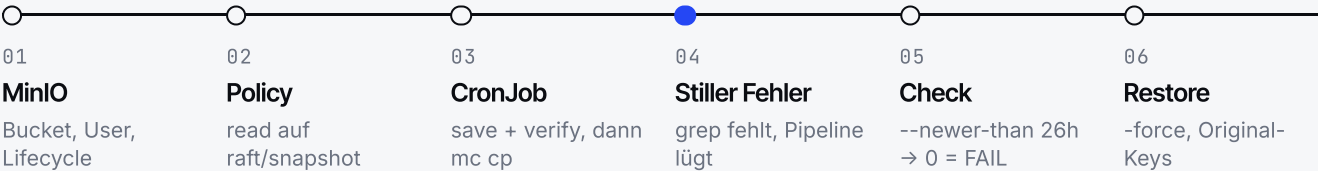
**Lease** – Lebensdauer eines dynamischen Secrets. Eine wiederhergestellte Instanz hält alle Leases des Originals und würde sie ablaufen lassen.

# Der Job meldete Erfolg. Seine Retention hatte nie funktioniert.

Dieser Leitfaden baut das Backup für den OpenBao aus N° 1: ein CronJob, der sich per ServiceAccount anmeldet, den Raft-Snapshot zieht, das Archiv nachrechnet und nach MinIO lädt; ein zweiter, der Alarm schlägt, wenn 26 Stunden lang keiner kam. Retention als Lifecycle-Regel auf dem Bucket – nicht im Job, aus gutem Grund.

Und eine Restore-Probe auf eine isolierte zweite Instanz, die den Punkt zeigt, den alle beim ersten Mal falsch verstehen: `restore -force` lässt die Instanz versiegelt zurück, der neue Key wird abgelehnt, nur der Original-Key entsiegelt. Mit dem echten Snapshot bis zu `Total Shares 5`.

```
-- Vier Schritte, zwei Key-Sätze
bao operator init                # Wegwerf-Keys
bao operator unseal <neuer key>
bao operator raft snapshot restore -force prod.snap
bao operator unseal <original-key> x3 # die neuen sind wertlos
```



|         |                                       |
|---------|---------------------------------------|
| openbao | 2.6.2 · raft · SA-Login               |
| minio   | chart 5.4.0 · versioning · ilm 14d/7d |
| mc      | RELEASE.2024-11-21 · kein grep        |
| drill   | openbao-drill · NetworkPolicy         |
| policy  | 1 Zeile · read                        |

20 Seiten · 5 Teile · 2 Anhänge  
Vertiefung: **Vault in Practice**  
[leanpub.com/vault-in-practice](https://leanpub.com/vault-in-practice)  
Gesetzt in Inter Tight, Inter und JetBrains Mono.  
Alle Ausgaben sind echte Terminal-Ausgaben.  
CC BY 4.0 · Weitergabe erlaubt · ohne Gewähr